

Haozhe Li

☎ +1 (437) 661-7680 | ✉ lihaozhe013@gmail.com | 📁 Portfolio | 🌐 lihaozhe013 | 🌐 lihaozhe013

EDUCATION

University of Toronto

B.A.Sc in Electrical & Computer Engineering + PEY Co-op

Toronto, ON, Canada

Sep. 2023 – Apr. 2028 (Expected)

Relevant Coursework: Communication Systems, Signals and Systems, Computer Hardware, Operating Systems, Digital Systems, Introductory Electronics, Circuit Analysis

TECHNICAL SKILLS

Programming Languages: C, C++, Python, RISC-V Assembly, Verilog, TypeScript

IDEs: STM32CubeIDE, Quartus Prime, ModelSim, DESim, Simulink, MATLAB, LTSpice, CPULator

Other Tools: Figma, Notion, Microsoft Office, L^AT_EX

Hardware Platforms: STM32 MCU, DE1-SoC FPGA, RISC-V RV32 DE1-SoC

Software Development: Node.js, React, Django, Vite, PyTorch, Git, Bash, Makefile, PostgreSQL, Nginx, GitHub Actions, Docker

EXPERIENCE

Full Stack Engineer (Contract)

Sep. 2025 – Present

Hangzhou MYF Technology Co., Ltd.

Remote

- Led requirements discovery and authored a comprehensive PRD; designed **Figma** wireframes and delivered a Node.js + SQLite MVP upon client sign-off.
- Developed a full-stack enterprise modules for tradeflow management using **TypeScript**, **Node.js**, and **PostgreSQL**.
- Used **GitHub Actions** to containerize and publish the application to **Docker Hub** via a **CI/CD** pipeline.
- Established an **Argo CD** pipeline for a **Docker**-based architecture (Backend, **Nginx**, **PostgreSQL**) on **AWS** Lightsail, featuring automated backups and seamless delivery.

Frontend Developer (Intern)

May. 2025 – Aug. 2025

Hangzhou EagleCloud Security Technology Inc.

Hangzhou, Zhejiang, China

- Developed new front-end features for an **Electron**-based enterprise cybersecurity desktop application and implemented UI functions for an admin web console. Utilized **TypeScript**, **React**, **Ant Design**, within a 7-person front-end team.
- Utilized **Cursor** (AI IDE), optimized coding workflows by applying advanced prompt engineering methodologies.
- Worked within standardized development workflows, including **Gitflow branching** (resolving merges and conflicts across QA and pre-production), **GitHub Pull Requests** with assigned reviewers and feedback resolution, and **CI/CD integration** testing on Alibaba Cloud (Yunxiao) for build consistency.

PROJECTS

Dinosaur Game on STM32 MCU – Course Project | C, STM32, Embedded Systems

Mar. 2026

- Developed a 2D Dinosaur Game in **C** on **STM32F446ZE Nucleo Board**, implementing gameplay, driver for GPIO and I2C peripherals, double buffering video engine, and delivered a playable game in 3 weeks.
- Integrated a 4*3 keypad for real-time user control, utilizing **CPU interrupts** to achieve low-latency input handling.
- Developed a OS-independent real-time 2D graphics engine capable of simultaneously driving two SSD1306 OLED displays; Utilized double buffering and DMA, achieving low-latency, highly stable image transmission via the I2C interface

Runner Game (FPGA Board Game) – Course Project | C, RISC-V Assembly, FPGA Board, CPULator

Mar. 2025

- Developed a 2D runner game in **C** on a **DE1-SoC FPGA board**, implementing core game logic, PS/2 keyboard, audio components, and video components, and delivered a playable game in 3 weeks.
- Integrated a PS/2 keyboard for real-time user control, utilizing **CPU interrupts** to achieve low-latency input handling.
- Built a OS-independent 2D graphics engine capable of real time rendering of images and text at 60 FPS.
- Built a square wave synthesizer for real time sound effect generation and mixing.

Greedy Mouse Game – Course Project | Verilog, FPGA Board, ModelSim, Quartus Prime

Nov. 2024

- Designed and implemented a 2D Greedy Mouse game in **Verilog** for the **DE1-SoC FPGA board**, featuring interactive keyboard input and VGA graphics output.
- Developed a PS/2 keyboard controller for real-time user input, and integrated it with a finite state machine (FSM) and on-chip memory modules to drive VGA output, enabling smooth display of bitmap animations and enhancing gameplay visuals.
- Built a lightweight, OS-independent 2D graphics engine capable of rendering color images, animations, and text at up to 60 FPS.
- Used ModelSim and DESim for early-stage simulation and debugging; finalized the design using Quartus Prime for synthesis and deployment to the FPGA board, followed by iterative optimization.

GIS Route Optimization Application – Course Project | C++, GTK, Git, A*, Dijkstra

Jan. 2025 – Apr. 2025

- Developed a Geographic Information System (GIS) desktop application in **C++** with **GTK** on Mate Desktop as a course project in a 3-person team, implemented map rendering, geographical name search, shortest path and multi-stop path finding features.
- Leveraged **multithreading** to preprocess 4GB of raw coordinate data, converting it into structured point, line, and polygon formats for canvas rendering in under 50 seconds on a 16-core test machine.
- Implemented pathfinding algorithms including **A*** for shortest path computation, and **Dijkstra**, **multi-start greedy**, and **simulated annealing** for multi-stop route optimization, achieving 90% of the technical evaluation score.
- Integrated real-time traffic visualization by using **TomTom API** data and **libcurl** HTTP module.

TradeFlow System (Open Source) | TypeScript, Node.js, PostgreSQL, React

Jul. 2025 – Present

- Architected a full-stack trade management solution, featuring inventory and financial tracking plus sales analysis.
- Engineered a high-performance backend using **Typescript**, **Node.js** and **PostgreSQL**.
- Developed a responsive frontend with **React** and **Ant Design**, adopting a component-driven architecture to accelerate feature delivery and maintain code reusability.

License Plate Recognition Systems – Course Project | Python, PyTorch

Jan. 2026 – Apr. 2026

- Architected a deep learning based License Plate Recognition System, designed a **CNN+CRNN** neural network, wrote **PyTorch** code for training and inference, and completed the model training and testing.
- Constructed a **Python** pipeline to batch-generate, add noise and tile license plate images with background, automatically producing 10,000 training data points that include the full vehicle background.
- Established a comprehensive data generation and training pipeline using Python and uv as the package manager, and write complete project documentation to ensure that training results are highly reproducible.